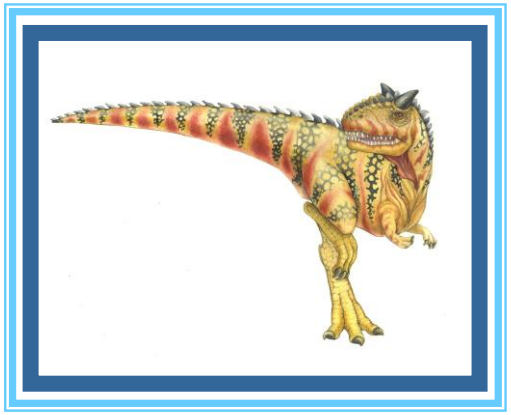
Chapter 2: Operating-System Services

فصل دوم: ساختارهای سیستم عامل



Silberschatz, Galvin and Gagne ©2013

Chapter 2: Outline

- Operating System Services
- User and Operating System-Interface
- System Calls
- System Services
- Linkers and Loaders
- Why Applications are Operating System Specific
- Design and Implementation
- Operating System Structure
- Building and Booting an Operating System
- Operating System Debugging



Operating System Services

- سیستم عامل محیطی فراهم می‌کند تا برنامه‌ها بتوانند اجرا شوند و کاربران از سیستم استفاده کنند.
- این خدمات به دو گروه کلی تقسیم می‌شوند:
  1. خدمات مفید برای کاربر (User-Oriented Services)
  2. خدمات داخلی برای مدیریت منابع (System-Oriented Services)



User-Oriented Services

1) رابط کاربری (User Interface – UI): تقریباً همه سیستم عامل‌ها یک رابط کاربری دارند که می‌تواند یکی از انواع زیر باشد:

- CLI (Command-Line Interface): رابط خط فرمان مثل Shell در لینوکس.
- GUI (Graphical User Interface): رابط گرافیکی مثل ویندوز.
- Batch Interface: اجرای گروهی دستورها بدون تعامل کاربر.

2) اجرای برنامه (Program Execution): سیستم عامل باید بتواند برنامه‌ای را از حافظه بارگذاری (load)، اجرا (run) و در پایان متوقف کند.



User-Oriented Services

3) عملیات ورودی/خروجی (I/O Operations): هر برنامه در حال اجرا ممکن است نیاز به استفاده از فایل یا دستگاه ورودی/خروجی داشته باشد.

4) مدیریت فایل‌ها (File System Manipulation): برنامه‌ها باید بتوانند فایل‌ها و پوشه‌ها را ایجاد (create)، حذف (delete)، جست‌وجو (search) و بخوانند یا بنویسند. همچنین مجوزها (permissions) باید قابل کنترل باشند.

5)) ارتباطات (Communication): فرایندها ممکن است نیاز داشته باشند با هم تبادل اطلاعات کنند، این ارتباط می‌تواند روی همان سیستم (Local Communication) یا بین سیستم‌ها از طریق شبکه (Network Communication) انجام شود.

- روش‌ها:
  - ✓ حافظه مشترک (Shared Memory)
  - ✓ ارسال پیام (Message Passing)



User-Oriented Services

6) تشخیص خطا (Error Detection): سیستم عامل باید همواره مراقب خطاها در CPU، حافظه، دستگاه‌های I/O یا برنامه‌ها باشد.

- برای هر نوع خطا، سیستم عامل اقدام مناسب را انجام می‌دهد تا ثبات (Consistency) سیستم حفظ شود.
- اشکال‌زدایی (Debugging) یکی از خدمات مهم است که به برنامه‌نویس اجازه می‌دهد خطاها را شناسایی و رفع کند.



System-Oriented Services

1) تخصیص منابع (Resource Allocation): در سیستم‌های چندکاربره یا چندپردازه‌ای، منابع (مثل CPU، حافظه، فایل‌ها) باید منصفانه بین همه تقسیم شوند.

2) حسابداری (Accounting): سیستم باید بداند هر کاربر چه مقدار از منابع سیستم استفاده کرده است (برای امنیت یا هزینه‌سنجی).

3) حفاظت و امنیت (Protection and Security): در سیستم‌های چندکاربره، باید کنترل شود که چه کسی به چه چیزی دسترسی دارد.

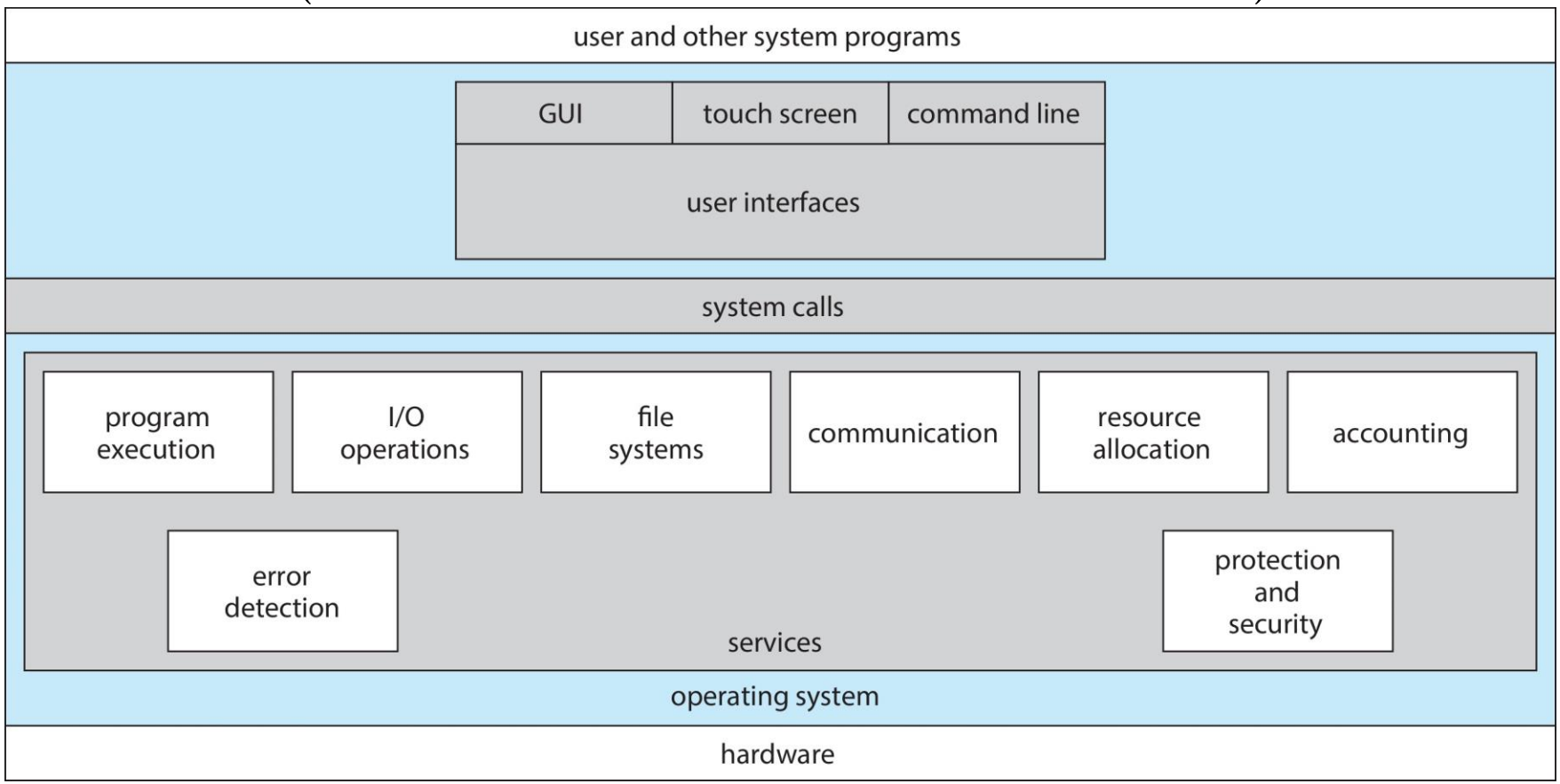
- Protection = کنترل دسترسی به منابع درون سیستم
- Security = محافظت از سیستم در برابر حملات بیرونی مثلاً احراز هویت کاربران
  - Authentication



A View of Operating System Services

کاربران ↔ رابط کاربری ↔ هسته سیستم عامل ↔ سخت‌افزار

(Users → UI → Kernel → Hardware)





Command Line interpreter -CLI

- CLIیا مفسر فرمان به کاربر اجازه می‌دهد مستقیماً فرمان وارد کند.
  - گاهی درون هسته (kernel) پیاده‌سازی می‌شود.
  - گاهی برنامه‌ای مجزا به نام Shellاست.
  - هر Shellدستورات را از کاربر می‌گیرد و اجرا می‌کند.



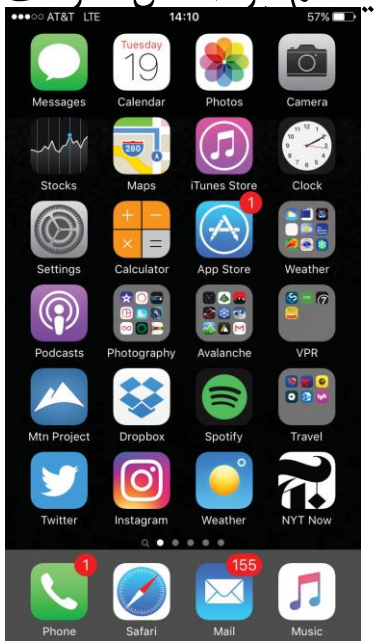
Graphical User Interface – GUI

- GUI یا رابط کاربر گرافیکی محیطی کاربرپسند (User-friendly) است.
- از ماوس، صفحه‌کلید و نمایشگر استفاده می‌کند.
- آیکون‌ها نماینده‌ی فایل‌ها، برنامه‌ها و دستورات هستند.
- کلیک روی آیکون باعث اجرای عملیات یا باز شدن پوشه می‌شود.
- امروزه سیستم‌های عامل مختلف معمولاً هر دو نوع رابط را دارند:
  - Windows: محیط GUI + پوسته‌ی خط فرمان (command shell)
  - Mac OS X: رابط Aqua GUI با هسته‌ی UNIX
  - Linux/Unix: رابط CLI با محیط‌های گرافیکی اختیاری مثل GNOME، KDE



Touchscreen Interfaces

- با گسترش گوشی‌ها و تبلت‌ها، رابط‌های کاربری مبتنی بر لمس (Touch-based Interfaces) ایجاد شدند.
- در این دستگاه‌ها استفاده از ماوس ممکن نیست، بنابراین تعامل کاربر با سیستم بر اساس حرکات (Gestures) است.

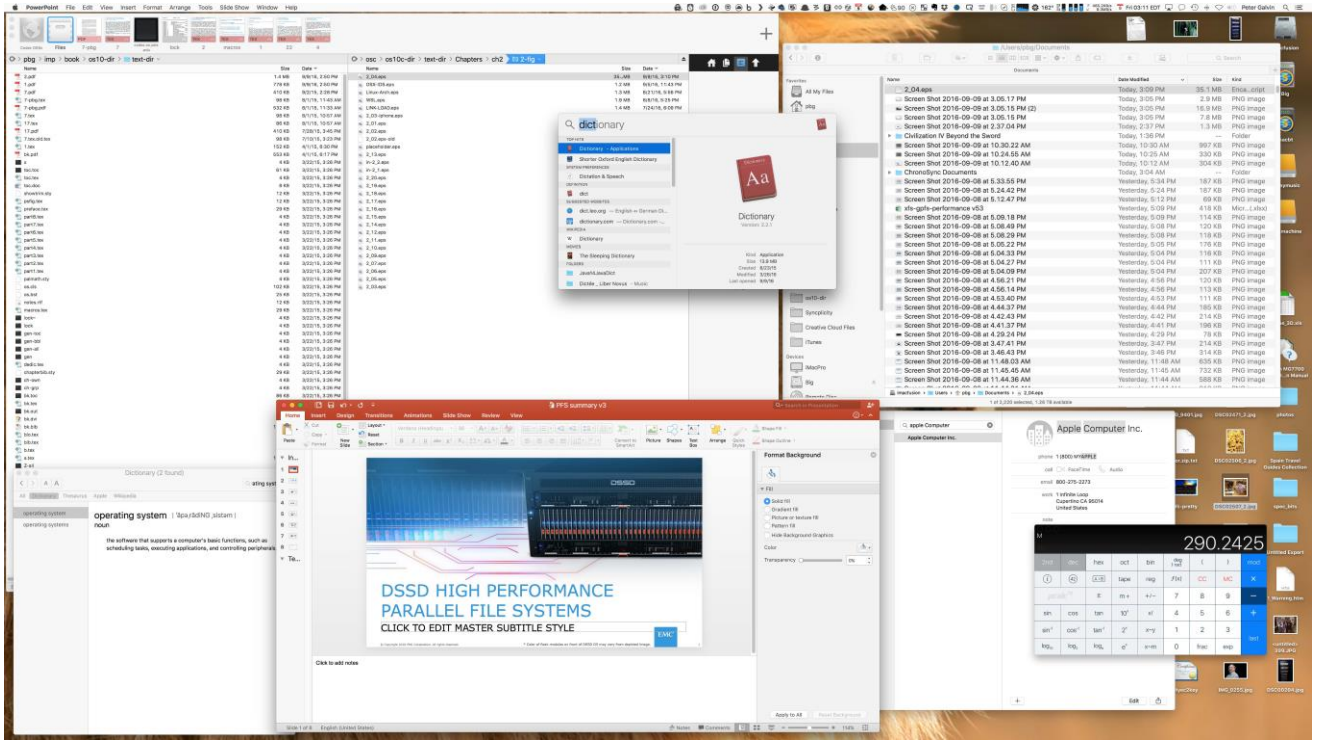


- ویژگی‌ها:
  - انتخاب و فرمان‌ها با لمس صفحه انجام می‌شوند.
  - از صفحه‌کلید مجازی (Virtual Keyboard) برای ورود متن استفاده می‌شود.
- در سیستم‌عامل Android یا iOS، کشیدن انگشت روی صفحه (Swipe) یا لمس طولانی (Long Press) جایگزین کلیک ماوس شده است.



The Mac OS X GUI

- محیط Aqua GUI از سیستم‌عامل مک (Mac OS X)
- Mac OS Xنمونه‌ای از سیستم‌عاملی است که هم رابط گرافیکی مدرن دارد و هم قدرت و پایداری هسته‌ی UNIXرا حفظ کرده است.



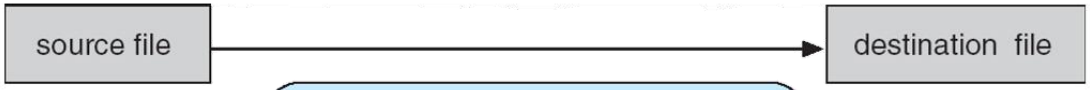
System Calls

- System Call یعنی درخواستی که یک برنامه از سیستم عامل برای انجام یک خدمت (Service) خاص ارسال می‌کند.
- تعریف: رابط برنامه‌نویسی بین برنامه‌های کاربر و خدمات سیستم عامل است.
- ویژگی‌ها:
  - معمولاً باش زبان‌های سطح بالا مثل C یا C++ نوشته می‌شوند.
  - اغلب از طریق رابط برنامه نویسی کاربردی API فراخوانی می‌شوند، نه مستقیماً.
- API: Application Programming Interface
- سه API رایج:
  1) Win32 API برای ویندوز
  2) POSIX API برای سیستم‌های شبه یونیکس (UNIX/Linux/Mac OS X)
  3) Java API برای ماشین مجازی جاوا (JVM)



System Calls

- برنامه‌نویسان معمولاً مستقیماً system call را نمی‌نویسند؛ بلکه از طریق توابع کتابخانه‌ای استاندارد مثل `printf()`, `open()`, `read()` با سیستم عامل ارتباط می‌گیرند.
- مثال: نمونه‌ای از توالی فراخوانی‌های سیستمی برای "کپی کردن محتویات یک فایل به فایل دیگر" نشان می‌دهد.



Example System Call Sequence

```
Acquire input file name
  Write prompt to screen
  Accept input
Acquire output file name
  Write prompt to screen
  Accept input
Open the input file
  if file doesn't exist, abort
Create output file
  if file exists, abort
Loop
  Read from input file
  Write to output file
Until read fails
Close output file
Write completion message to screen
Terminate normally
```

مراحل:

1. باز کردن فایل مبدأ (open source file)
2. ایجاد فایل مقصد (create destination file)
3. خواندن داده‌ها از فایل مبدأ (read)
4. نوشتن داده‌ها در فایل مقصد (write)
5. بستن هر دو فایل (close)



System Calls

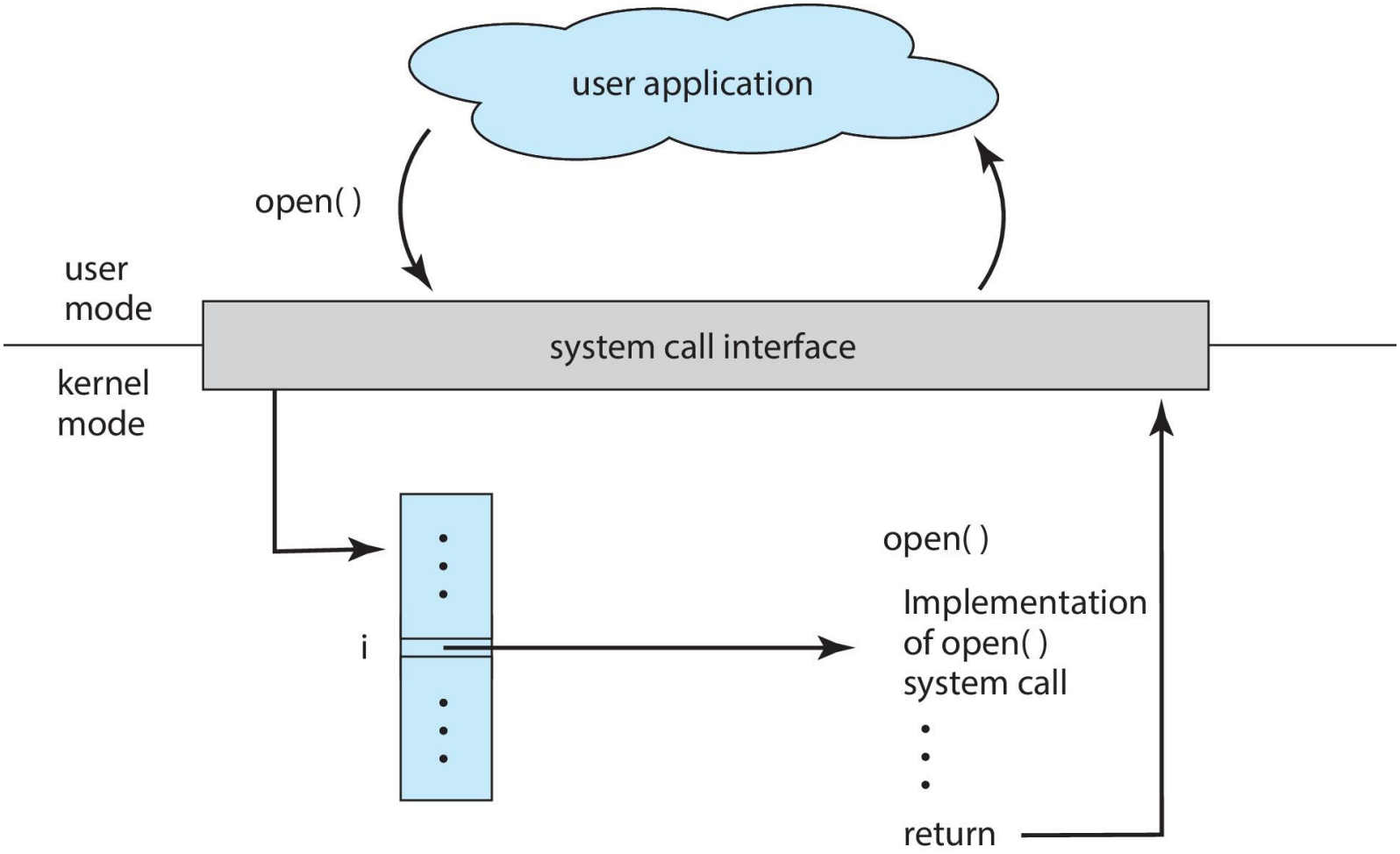
- مثال در C:

```
open("file1.txt");
create("file2.txt");
while (read(...) != 0)
    write(...);
close(...);
```

- تابع `fopen()` → فراخوانی `open()` در سطح سیستم عامل
- تابع `printf()` → فراخوانی `write()` در سطح سیستم عامل
-
- API مثل یک «واسط کاربرپسند» بین برنامه‌نویس و هسته‌ی سیستم عامل است.



API – System Call – OS Relationship





System Call Implementation

- هر system call دارای شماره‌ای منحصربه‌فرد (System Call Number) است.
- سیستم عامل جدولی از همه system callها نگه می‌دارد (System Call Table).
- وقتی برنامه فراخوانی انجام می‌دهد، شماره‌ی آن به هسته (Kernel) فرستاده می‌شود.

1) System Call Interface آن شماره را در جدول جستجو می‌کند.
2) تابع متناظر در هسته را اجرا می‌کند.
3) نتیجه یا کد خطا را به برنامه‌ی فراخوان بازمی‌گرداند.

- برنامه‌نویس لازم **نیست** بداند درون سیستم عامل چه اتفاقی می‌افتد؛ فقط باید بداند چه چیزی درخواست می‌کند (از طریق API).



System Call Parameter Passing

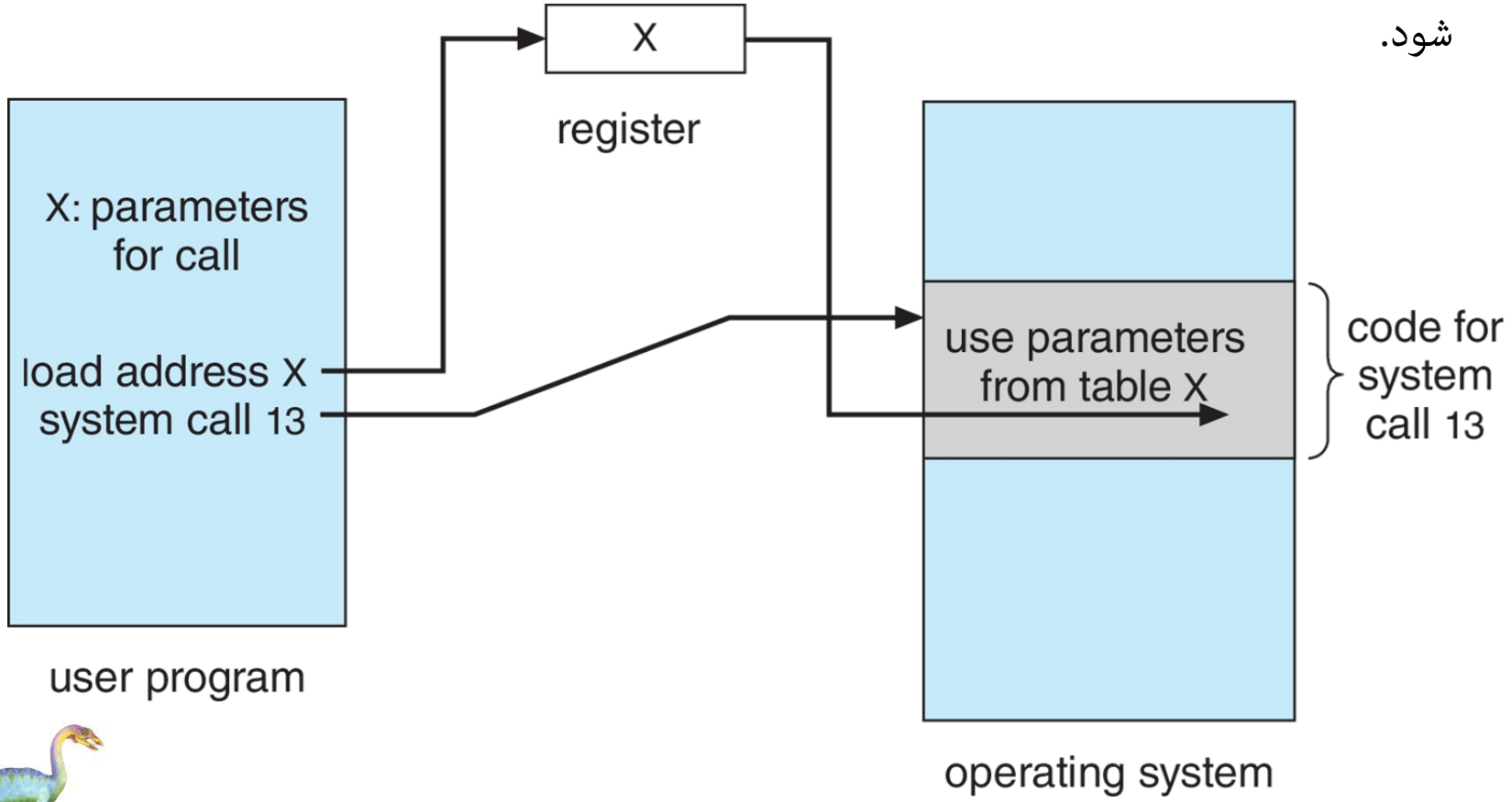
- در بسیاری از system callها باید اطلاعاتی مثل نام فایل، اندازه بافر یا داده‌های ورودی به سیستم عامل ارسال شود؛ این اطلاعات به نام پارامترها شناخته می‌شوند.
- سه روش اصلی برای انتقال پارامترها:

1) در ثبات‌ها (Registers):
   - پارامترها مستقیماً در ثبات‌های CPU قرار داده می‌شوند.
   - مناسب برای تعداد کم پارامترها.
2) در بلوک حافظه (Memory Block)
   - پارامترها در یک جدول یا بلوک در حافظه ذخیره می‌شوند و آدرس آن به OS داده می‌شود.
   - در سیستم‌عامل‌های Linux و Solaris رایج است
3) روی پشته (Stack)
   - پارامترها روی پشته قرار گرفته و OS آن‌ها را از پشته می‌خواند.
   - روش عمومی و بدون محدودیت در تعداد پارامترها



Parameter Passing via Table

- پارامترها در یک جدول در حافظه قرار می‌گیرند و فقط آدرس جدول به عنوان پارامتر به سیستم عامل ارسال می‌شود.
- این روش ساده‌تر و ایمن‌تر است چون نیازی نیست تعداد زیادی مقدار در ثبات‌ها یا پشته کپی شود.





Types of System Calls

- سیستم‌عامل‌ها مجموعه‌ای از System Callها دارند که در چند گروه اصلی دسته‌بندی می‌شوند.

1) کنترل فرایند (Process Control):
   - مثل create, terminate ،load, execute ،end, abort ,lock،process debugger، allocate/free memory
   - در لینوکس وقتی می‌نویسیم `fork()`، سیستم عامل یک فرایند جدید می‌سازد. این یکی از مهم‌ترین فراخوانی سیستمی های گروه Process Controlاست.
2) مدیریت فایل (File Management):
   - فراخوانی‌هایی که برای کار با فایل‌ها استفاده می‌شوند مثل create file، delete file، close file، Read، Write
3) مدیریت دستگاه (Device Management):
   - فراخوانی‌هایی برای کنترل دستگاه‌های سخت‌افزاری (I/O devices) مثل request device, release device,Read,Write
   - درایور چاپگر (Printer Driver) با system callها کار می‌کند تا داده را به چاپگر ارسال کند یا وضعیت آن را بررسی نماید.



Types of System Calls

4) نگهداری اطلاعات (Information Maintenance):

- این گروه برای مدیریت داده‌های کلی سیستم است مثل get time or date, set time or dateget/set process, file, or device attributes

5) ارتباطات (Communications):

- برای تبادل داده میان فرایندها یا میان رایانه‌ها در شبکه مثل create/delete communication connection، shared memory یا transfer status information

6) حفاظت (Protection):

- هدف این گروه کنترل دسترسی کاربران و فرآیندها به منابع است مثل control access to resources،allow/deny user access

- در لینوکس دستور `chmod` برای تغییر سطح دسترسی فایل‌ها از system call سطح پایین‌تر استفاده می‌کند.



Examples of Windows and Unix System Calls

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

The following illustrates various equivalent system calls for Windows and UNIX operating systems.

	Windows	Unix
Process control	`CreateProcess()`	`fork()`
	`ExitProcess()`	`exit()`
	`WaitForSingleObject()`	`wait()`
File management	`CreateFile()`	`open()`
	`ReadFile()`	`read()`
	`WriteFile()`	`write()`
	`CloseHandle()`	`close()`
Device management	`SetConsoleMode()`	`ioctl()`
	`ReadConsole()`	`read()`
	`WriteConsole()`	`write()`
Information maintenance	`GetCurrentProcessID()`	`getpid()`
	`SetTimer()`	`alarm()`
	`Sleep()`	`sleep()`
Communications	`CreatePipe()`	`pipe()`
	`CreateFileMapping()`	`shm_open()`
	`MapViewOfFile()`	`mmap()`
Protection	`SetFileSecurity()`	`chmod()`
	`InitlializeSecurityDescriptor()`	`umask()`
	`SetSecurityDescriptorGroup()`	`chown()`

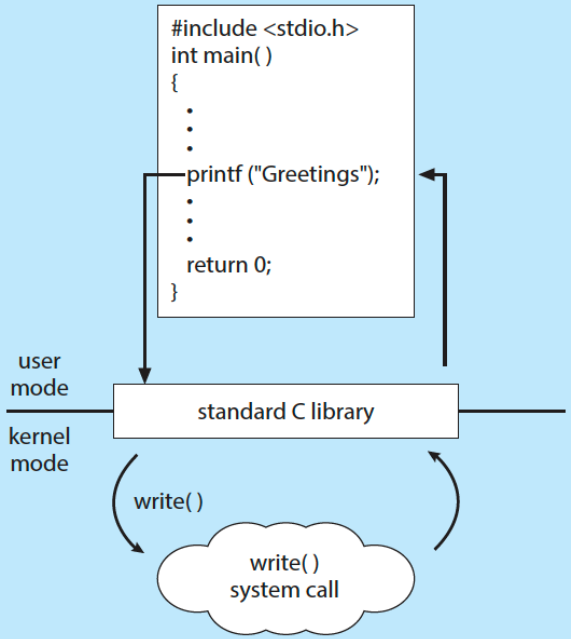


Standard C Library Example

- در زبان C وقتی تابع `printf()` را فراخوانی می‌کنید، در واقع این تابع خودش از فراخوانی سیستمی `write()` استفاده می‌کند تا خروجی را به دستگاه (مثلاً صفحه‌نمایش) بفرستد.

THE STANDARD C LIBRARY

The standard C library provides a portion of the system-call interface for many versions of UNIX and Linux. As an example, let's assume a C program invokes the `printf()` statement. The C library intercepts this call and invokes the necessary system call (or calls) in the operating system—in this instance, the `write()` system call. The C library takes the value returned by `write()` and passes it back to the user program:





System Services

- برنامه‌های سیستمی ابزارهایی هستند که محیطی مناسب برای توسعه و اجرای برنامه‌ها فراهم می‌کنند.
- این برنامه‌ها خودشان از System Callها استفاده می‌کنند.
- گروه‌های اصلی:
  1) File Manipulation → کار با فایل‌ها
  2) Status Information → نمایش اطلاعات وضعیت سیستم
  3) Programming Language Support → پشتیبانی از زبان‌های برنامه‌نویسی
  4) Program Loading and Execution → بارگذاری و اجرای برنامه
  5) Communications → برقراری ارتباط
  6) Background Services → سرویس‌های پس‌زمینه
  7) Application Programs → برنامه‌های کاربردی
- کاربران معمولاً از طریق همین برنامه‌های سیستمی با سیستم عامل تعامل دارند، نه مستقیماً با system callها.



System Services

1) مدیریت فایل‌ها (File Management)

- شامل ابزارهایی برای ایجاد، حذف، کپی، تغییر نام، چاپ، و فهرست‌کردن فایل‌ها و دایرکتوری‌ها است.

2) اطلاعات وضعیت (Status Information)

- برخی از برنامه‌ها اطلاعات کلی مانند: زمان و تاریخ/مقدار حافظه‌ی آزاد/ فضای دیسک/ تعداد کاربران فعال را نشان میدهند.
- دیگر برنامه‌ها گزارش عملکرد و اشکال‌زدایی (logging and debugging) ارائه می‌کنند.
- در ویندوز این بخش شامل ابزار Event Viewerاست.
- رجیستری (Registry):در برخی سیستم‌ها (مثل Windows) داده‌های پیکربندی در Registry نگهداری می‌شود.



System Services

3) ویرایش فایل‌ها (File Modification)

- ابزارهایی مثل ویرایشگر متن (Text Editor) برای ساخت یا ویرایش فایل‌ها
- ابزارهای جست‌وجوی محتوا یا انجام تغییرات خودکار در متن (مثل `grep` یا `sed` در UNIX)

4) پشتیبانی از زبان برنامه‌نویسی (Programming-Language Support)

- کامپایلرها (Compilers)
- اسمبلرها (Assemblers)
- مفسرها (Interpreters)
- خطایاب‌ها (Debuggers)



System Services

4) بارگذاری و اجرای برنامه (Program Loading and Execution)

- Loaderها (برای بارگذاری برنامه‌ها در حافظه)
- Linkerها (برای اتصال کدها و کتابخانه‌ها)
- Debuggerها (برای تست اجرای برنامه‌ها)

5) ارتباطات (Communications)

- ارسال پیام بین کاربران (Messaging)
- مرور وب (Browsing)
- ارسال ایمیل (Email)
- ورود از راه دور (Remote Login)
- انتقال فایل (File Transfer)

- در واقع بیشتر کاربران فکر می‌کنند این برنامه‌ها خود سیستم عامل هستند، در حالی که فقط ابزارهایی هستند که از خدمات سیستم عامل استفاده می‌کنند.



System Services

6) سرویس‌های پس‌زمینه (Background Services)

- برخی هنگام راه‌اندازی سیستم (Boot time) اجرا می‌شوند.
- بعضی فقط برای آماده‌سازی سیستم و برخی تا خاموش شدن سیستم فعال می‌مانند.
- کارهایی مثل: بررسی دیسک (Disk Checking)، زمان‌بندی فرایندها (Process Scheduling)، ثبت خطا (Error Logging) و چاپ (Printing).
- این سرویس‌ها در سطح کاربر اجرا می‌شوند و به آنها Daemon (در یونیکس) یا Service (در ویندوز) گفته می‌شود.

7) برنامه‌های کاربردی (Application Programs)

- مستقیماً توسط کاربران اجرا می‌شوند.
- مثل مرورگرها، ویرایشگرها، یا نرم‌افزارهای آفیس.
- معمولاً بخشی از سیستم‌عامل محسوب نمی‌شوند اما از خدمات آن استفاده می‌کنند.




File Manipulation
Application Programs
Status Information
System Programs
Background Services
Programming Support
Communications
Execution

Design and Implementation

- دو دیدگاه و هدف در طراحی OS:
  - User Goals (اهداف کاربر): سیستم عامل باید استفاده راحت، یادگیری آسان، قابل اعتماد، ایمن و سریع باشد.
  - System Goals(اهداف سیستم): سیستم عامل باید طراحی، پیاده‌سازی و نگهداری آسان و همچنین انعطاف‌پذیر، قابل اعتماد، بدون خطا و کارآمد باشد.

- تعیین و طراحی یک سیستم عامل یک کار بسیار خلاقانه مهندسی نرم‌افزار است.



مفهوم Policy و Mechanism در طراحی سیستم‌عامل

- در طراحی سیستم عامل، اصل مهمی وجود دارد:
  - Policy سیاست: چه کاری انجام شود؟ تصمیماتی که رفتار کلی سیستم را مشخص می‌کنند. مثلا : کدام فرایند باید اجرا شود؟
  - Mechanism: چگونه انجام شود؟ ابزار یا روش فنی برای اجرای سیاست . مثلا الگوریتم زمان‌بندی

- Policy: تصمیم Mechanism: ابزار اجرای تصمیم
- تفکیک این دو باعث انعطاف‌پذیری و قابلیت نگهداری بیشتر سیستم‌عامل می‌شود.
- تغییر در سیاست‌ها نباید نیاز به بازنویسی کل مکانیزم‌ها داشته باشد.
- مثال:
  - مکانیزم: چگونگی تخصیص CPU به فرآیندها.
  - سیاست: تصمیم‌گیری درباره‌ی اینکه کدام فرآیند در اولویت است.



مثال از Policy و Mechanism در سیستم‌عامل

- مثال ۱ : زمان‌بندی CPU (CPU Scheduling)

  Policy تعیین شود کدام فرآیند اول اجرا شود (مثلاً براساس اولویت).

  Mechanism استفاده از الگوریتمی مثل Round Robin یا Priority برای اجرای آن تصمیم Scheduling.

- مثال ۲ – کنترل دسترسی فایل‌ها (File Access Control)

  Policy فقط کاربران مجاز بتوانند فایل را بخوانند.

  Mechanism استفاده از ACL (Access Control List) یا Permission Bits برای اجرای سیاست امنیتی.

✓ اگر Policy و Mechanism در طراحی سیستم از هم جدا باشند، می‌توان سیاست‌ها را به‌سادگی تغییر داد بدون اینکه نیاز به بازنویسی ساختار فنی (Kernel) باشد.

✓ مکانیزم معمولاً در **داخل هسته** پیاده‌سازی می‌شود، ولی سیاست Policy معمولاً **خارج از هسته** یا قابل تغییر در زمان اجرا طراحی می‌شود.



Implementation

- در سیستم‌عامل‌ها روش‌های پیاده‌سازی متفاوتی وجود دارد.
  - در ابتدا سیستم‌عامل‌ها با زبان Assembly نوشته می‌شدند.
  - بعدها از زبان‌های برنامه‌نویسی سیستمی مانند Algol و PL/1 استفاده شد.
  - امروزه بیشتر سیستم‌عامل‌ها با زبان‌های C و C++ نوشته می‌شوند.
- در واقع معمولاً ترکیبی از چند زبان به‌کار می‌رود:
  - قسمت‌های پایین‌تر و وابسته به سخت‌افزار در Assembly،
  - بدنه‌ی اصلی سیستم در C،
  - و برنامه‌های سیستمی در زبان‌هایی مثل C، C++، Python، PERL یا Shell Script نوشته می‌شوند.
- زبان‌های سطح بالا باعث می‌شوند سیستم‌عامل راحت‌تر به سخت‌افزارهای دیگر منتقل شود (Portable)
  - اما ممکن است اجرای آن کمی کندتر باشد.
- گاهی از شبیه‌سازی (Emulation) استفاده می‌شود تا سیستم‌عامل روی سخت‌افزار غیر بومی نیز اجرا شود.



Operating System Structure

- سیستم‌عامل‌های عمومی برنامه‌های بسیار بزرگی هستند که از بخش‌های مختلف تشکیل شده‌اند.
- برای درک و مدیریت این پیچیدگی، از روش‌های گوناگونی برای ساختاردهی سیستم‌عامل استفاده می‌شود.
- روش‌های اصلی ساختار سیستم‌عامل:
  - **ساختار ساده (Simple Structure)** مانند MS-DOS که بخش‌ها تفکیک زیادی ندارند.
  - **ساختار پیچیده‌تر (More Complex)** مانند UNIX که ماژول‌ها و وظایف جداگانه‌تری دارد.
  - **ساختار لایه‌ای (Layered Structure)** که در آن سیستم به چند لایه تقسیم می‌شود و هر لایه فقط با لایه‌های نزدیک خود در ارتباط است.
  - **ساختار میکروهسته‌ای (Microkernel)** مانند Mach که بیشتر بخش‌ها را از هسته جدا کرده و به فضای کاربر منتقل می‌کند تا سیستم قابل توسعه‌تر و ایمن‌تر باشد.
- هدف از این ساختاردهی، ساده‌تر شدن طراحی، نگهداری و توسعه سیستم‌عامل است.



Simple Structure

- سیستم‌عامل MS-DOS نمونه‌ای از ساختار ساده است.
- در آن همه بخش‌ها تقریباً در یک سطح قرار دارند و مرز مشخصی بین لایه‌ها وجود ندارد.
- رابط کاربر (Command Interpreter)، سیستم فایل (File System) و کنترل دستگاه‌ها (Device Drivers) همگی مستقیماً با سخت‌افزار در ارتباط‌اند.
- در MS-DOS، چون برای رایانه‌های شخصی کوچک طراحی شده بود، سادگی و حجم کم مهم‌تر از نظم و تفکیک بود.
- به همین دلیل همه بخش‌ها در کنار هم در یک فضای حافظه اجرا می‌شوند.
- مثلاً خطای یک برنامه ممکن است باعث از کار افتادن کل سیستم شود، چون هیچ جداسازی واقعی بین اجزا وجود ندارد.



More Complex System

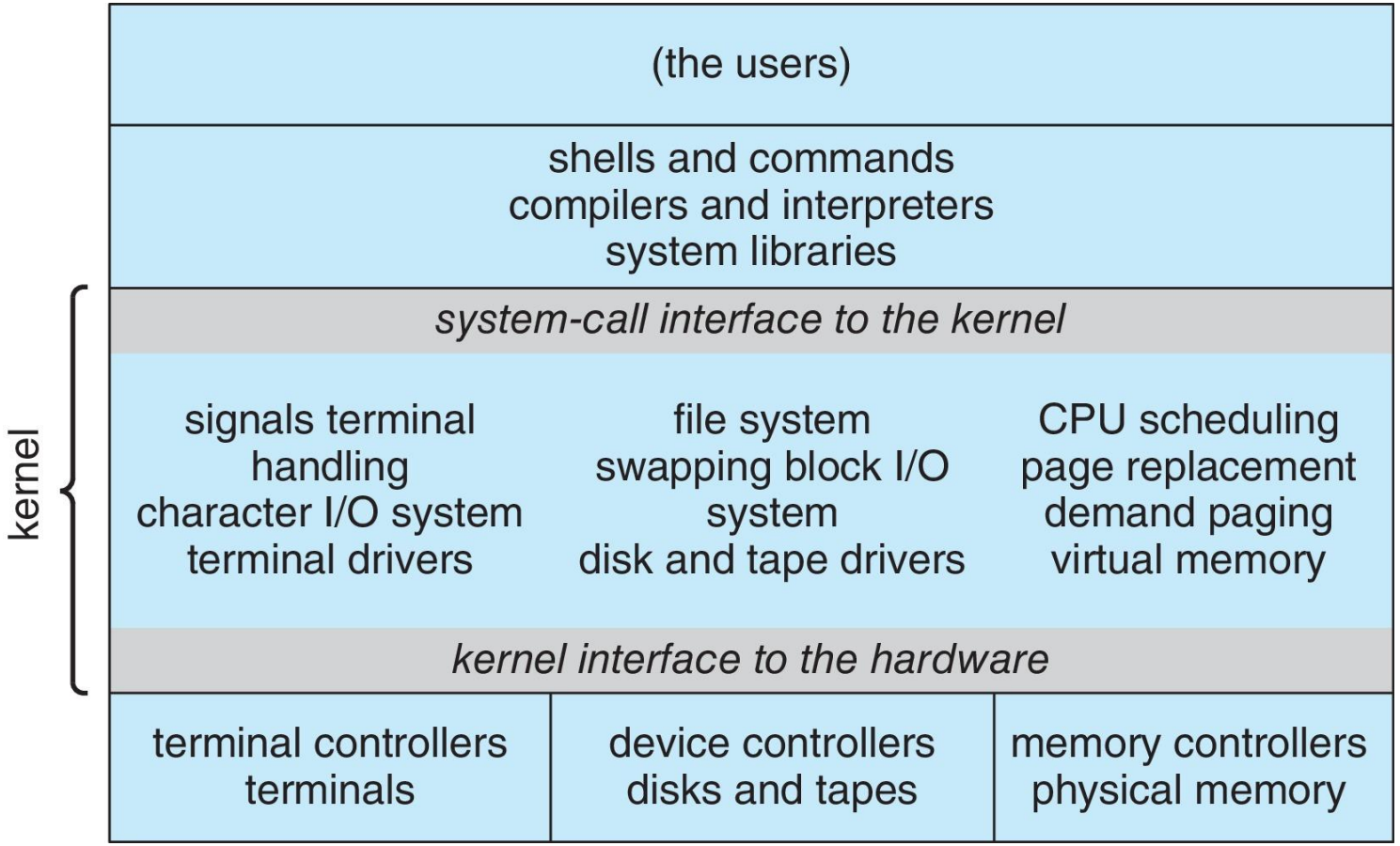
- سیستم‌عامل UNIX ساختار منظم‌تری نسبت به MS-DOS دارد.
- به‌صورت کلی از دو بخش تشکیل شده است:
  1) System Programs
  2) Kernel

- Kernel شامل بخش‌های اصلی مانند زمان‌بندی CPU، مدیریت حافظه، فایل‌سیستم و عملیات ورودی/خروجی است.
- System Programsابزارهایی برای کاربران و برنامه‌نویسان فراهم می‌کنند.
- در UNIX، مرز روشنی بین سطح کاربر (User Mode) و سطح هسته (Kernel Mode) وجود دارد.
- این جداسازی باعث پایداری و امنیت بیشتر سیستم می‌شود.
- اما هنوز سیستم به طور کامل لایه‌ای نیست؛ برخی قسمت‌ها هنوز به‌صورت مستقیم با هم تعامل دارند.



Traditional UNIX System Structure

فراتر از ساده اما نه کاملاً لایه لایه



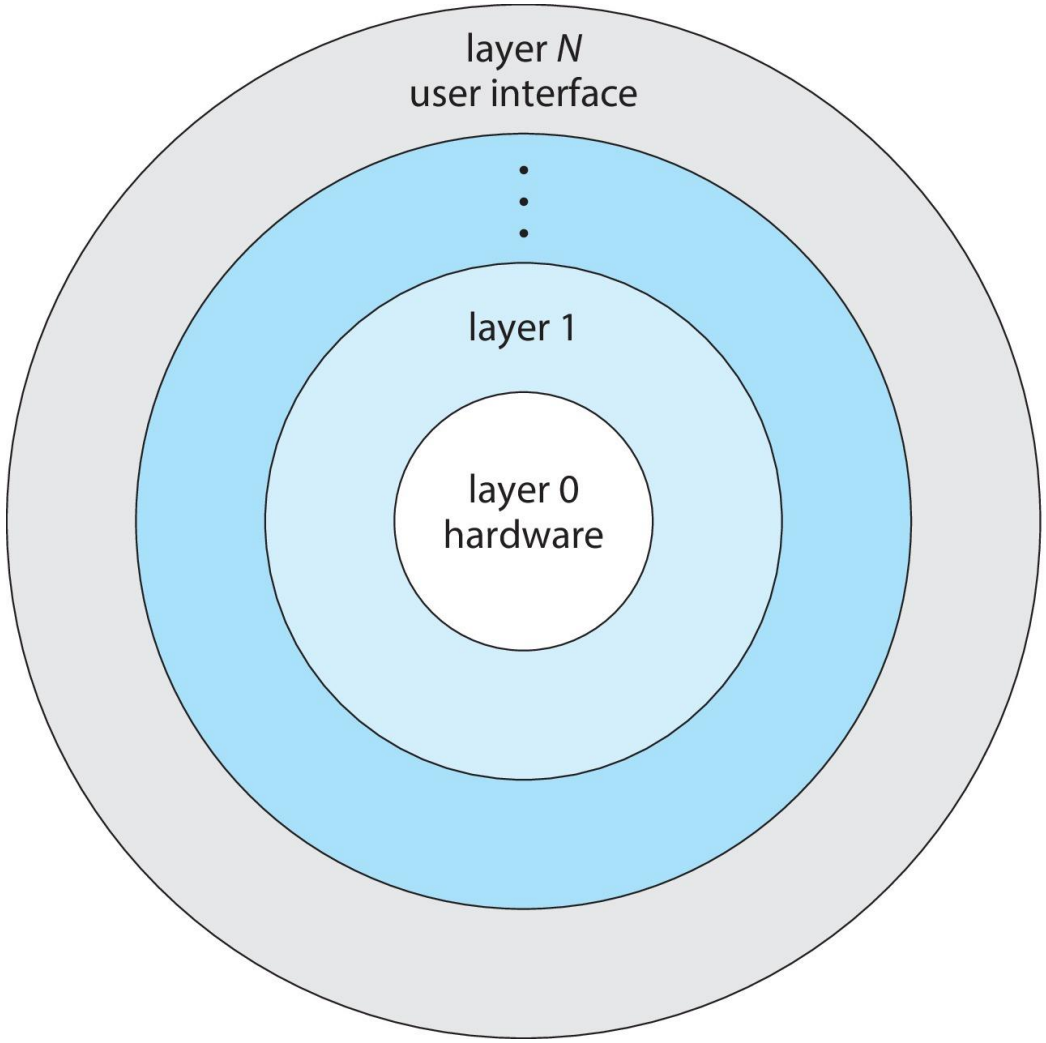


Layered Structure

- در ساختار لایه‌ای، سیستم‌عامل از چند لایه تشکیل می‌شود.
- پایین‌ترین لایه سخت‌افزار است و بالاترین لایه رابط کاربر یا برنامه‌های کاربردی است.
- هر لایه فقط از خدمات لایه‌ی پایین‌تر استفاده می‌کند.
- مثلاً:
  - لایه 0: سخت‌افزار
  - لایه 1: مدیریت حافظه و دستگاه‌ها
  - لایه 2: زمان‌بندی و فایل‌سیستم
  - لایه 3: رابط کاربر
- این مدل باعث می‌شود طراحی، تست و اشکال‌زدایی سیستم آسان‌تر شود،
- چون هر لایه مستقل از دیگری عمل می‌کند.
- یکی از اولین سیستم‌های لایه‌ای، سیستم THE بود که توسط Dijkstra طراحی شد.
- عیب اصلی آن این است که اگر تعداد لایه‌ها زیاد شود، ارتباط بین آن‌ها کندتر می‌شود.



Layered Structure



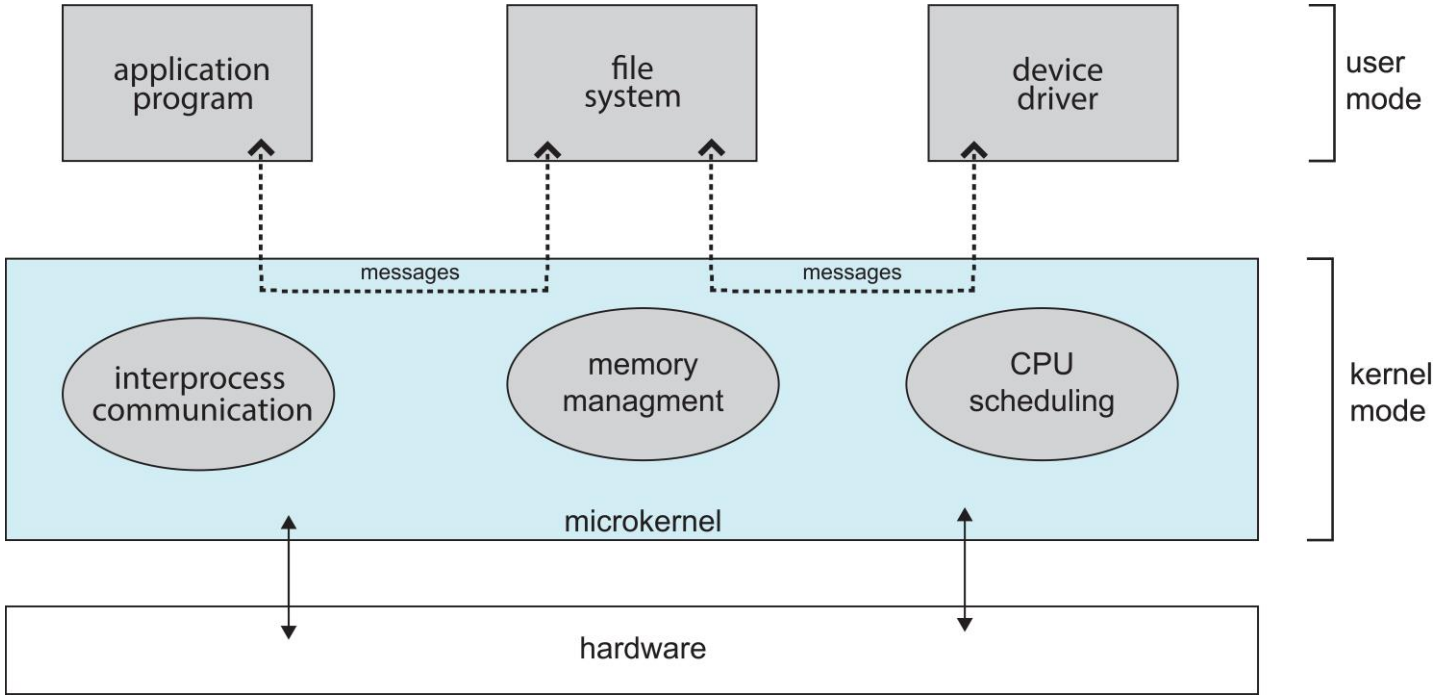


Microkernel Structure

- در ساختار میکروهسته‌ای، بخش عمده‌ای از وظایف از هسته خارج می‌شود و در فضای کاربر اجرا می‌شود.
- هسته فقط کارهای اصلی را انجام می‌دهد:
  - ارتباط بین پردازه‌ها(Interprocess Communication)
  - مدیریت پایه‌ای حافظه
  - زمان‌بندیCPU
- دیگر سرویس‌ها مانند فایل‌سیستم یا درایورها در خارج از هسته قرار دارند و از طریق پیام (Message Passing)با هسته ارتباط می‌گیرند.
- در سیستم‌هایی مثل Mach (پایهMac OS X) یا QNX این مدل استفاده می‌شود.
- مزیت آن این است که سیستم پایدارتر و قابل توسعه‌تر است؛ اگر یکی از ماژول‌ها خراب شود، کل سیستم از کار نمی‌افتد.
- اما چون ارتباط بین هسته و ماژول‌ها از طریق ارسال پیام انجام می‌شود، کمی کاهش کارایی وجود دارد.



Microkernel System Structure

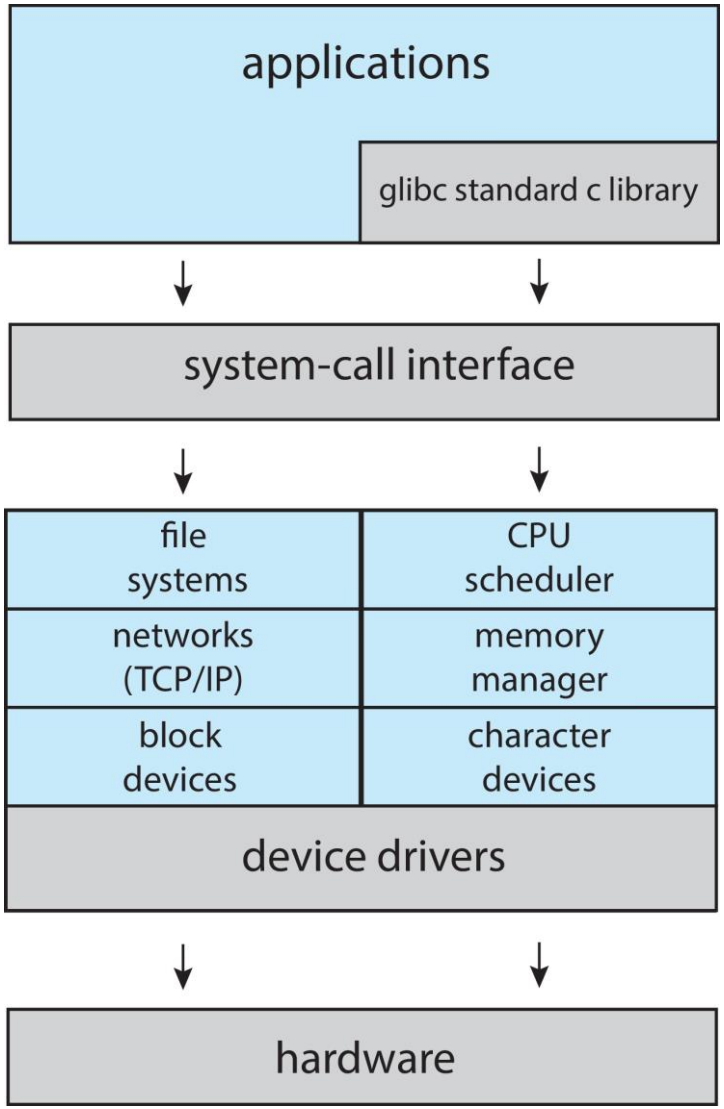


ساختار ماژولار (Modules)

- اکثر سیستم‌عامل‌های مدرن از ماژول‌های بارگذاری‌شونده (Loadable Kernel Modules – LKM) استفاده می‌کنند.
- ویژگی‌ها:
  - هر مؤلفه (Component) مستقل است.
  - ارتباط بین مؤلفه‌ها از طریق رابط‌های مشخص انجام می‌شود.
  - فقط در زمان نیاز بارگذاری (Load) می‌شوند.
  - ترکیبی از مزایای ساختار لایه‌ای و میکروهسته‌ای است.
- مثال:
  - سیستم‌عامل‌های Linux
  - Solaris



Linux System Structure





Hybrid Systems

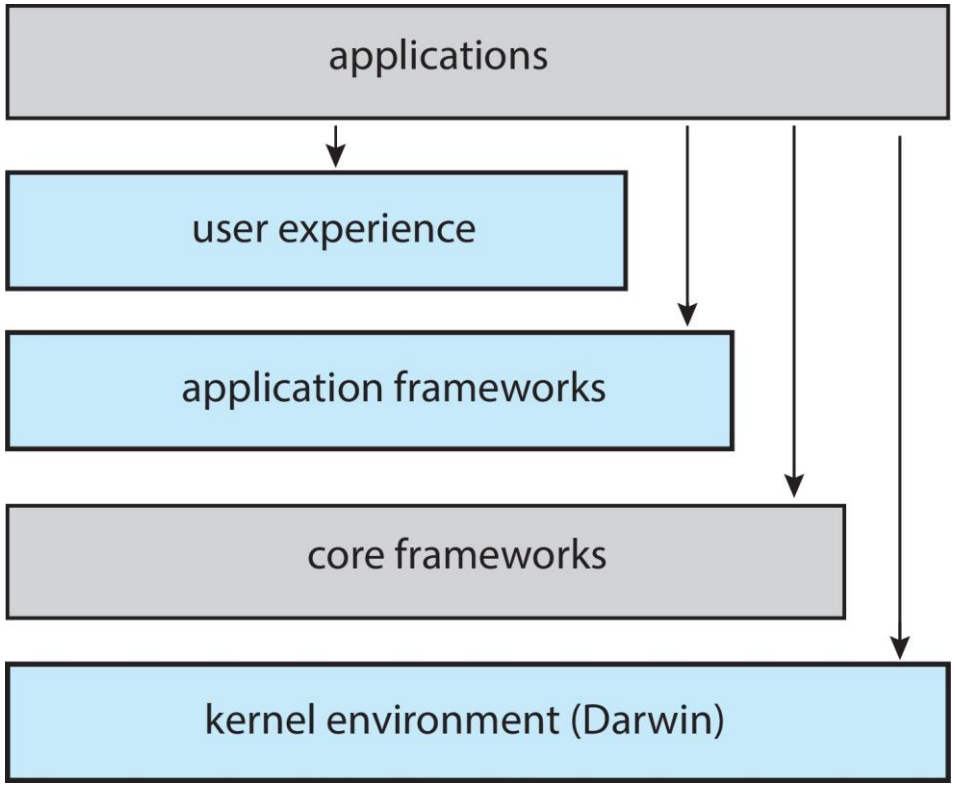
- بیشتر سیستم‌عامل‌های امروزی از ترکیب چند ساختار مختلف ساخته شده‌اند.به این نوع ساختار «سیستم ترکیبی» گفته می‌شود.
- در این مدل، طراحان تلاش می‌کنند بین کارایی بالا (Performance) و انعطاف‌پذیری (Flexibility) تعادل برقرار کنند.
- نمونه‌های معروف از سیستم‌های ترکیبی:Windows/ Solaris/ Linux/ macOS

- سیستم‌های ترکیبی در واقع بهترین ویژگی‌های هر ساختار را با هم ترکیب می‌کنند:
  - از مدل تک‌بلوک (Monolithic) سرعت بالا را می‌گیرند،
  - از مدل میکروهسته (Microkernel) پایداری و جداسازی مؤلفه‌ها را،
  - و از مدل ماژولار (Modular) قابلیت افزودن یا حذف بخش‌ها در زمان اجرا را.

- برای مثال در ویندوز، بخش‌های مختلف مانند مدیریت فایل، گرافیک، و شبکه در فضای کاربر جدا اجرا می‌شوند، اما ارتباط آن‌ها با هسته سریع و مستقیم است تا کارایی حفظ شود.



macOS and iOS Structure





- سیستم عامل Android توسط Open Handset Alliance (به رهبری گوگل) توسعه یافت.

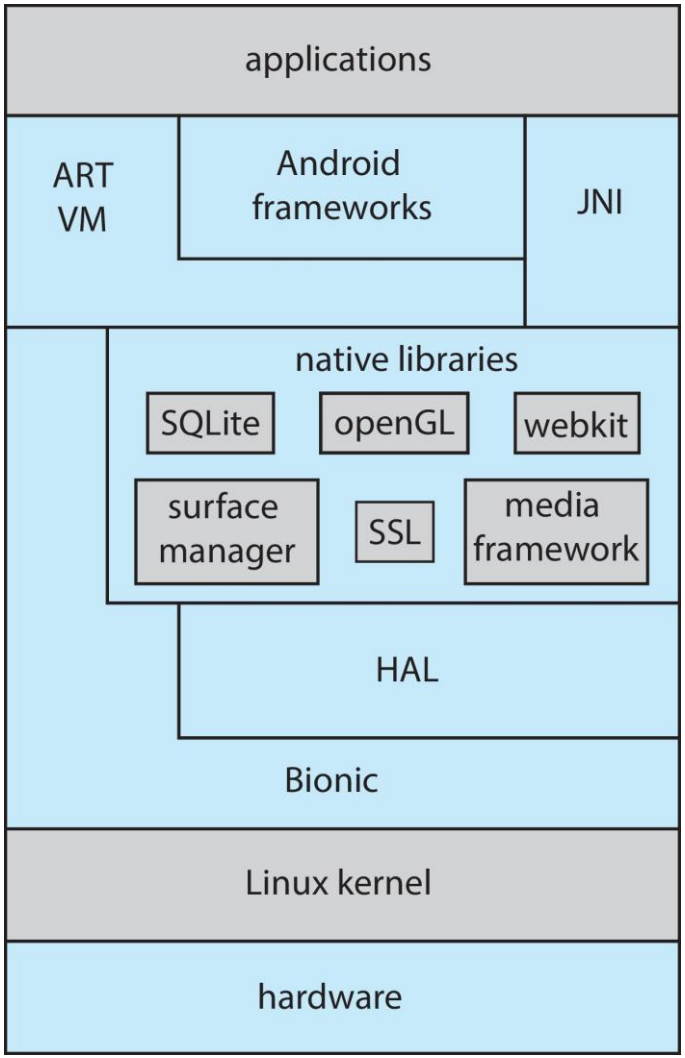
- ویژگی‌ها:
  - متن‌باز (Open Source)
  - مبتنی بر Linux Kernel با تغییرات اختصاصی
  - شامل مدیریت پردازه، حافظه، و درایورها
  - دارای Runtime Environment شامل کتابخانه‌ها و ماشین مجازی Dalvik VM
  - اپلیکیشن‌ها با Java + Android API توسعه می‌یابند.



Android Architecture

چهار لایه Android :

1. Applications
2. Application Framework
3. Libraries + Android Runtime
4. Linux Kernel





Building and Booting an Operating System

- سیستم‌عامل معمولاً برای یک رده از ماشین‌ها طراحی می‌شود، اما باید قبل از اجرا برای هر سیستم خاص تنظیم (Configure) شود.
- فرآیند آماده‌سازی و ساخت سیستم‌عامل را System Generation (SYSGEN) می‌گویند.
- در این مرحله اطلاعاتی درباره سخت‌افزار سیستم جمع‌آوری می‌شود، مثل:
  - نوع پردازنده و حافظه
  - دستگاه‌های ذخیره‌سازی
  - درایورها و ورودی/خروجی‌ها
- با استفاده از این اطلاعات، نسخه‌ای از هسته ساخته می‌شود که دقیقاً با سخت‌افزار موجود سازگار است.نتیجه‌ی این کار یک سیستم‌عامل بهینه‌شده برای آن دستگاه است.
- در گذشته باید این کار به‌صورت دستی انجام می‌شد (مثلاً در UNIX)،
- اما امروزه بیشتر سیستم‌عامل‌ها در زمان نصب این کار را به‌صورت خودکار انجام می‌دهند.
- مثل مرحله نصب لینوکس یا ویندوز که سیستم‌عامل سخت‌افزار را شناسایی و کرنل مناسب را می‌سازد.



System Boot

- وقتی کامپیوتر روشن می‌شود، فرآیند Booting آغاز می‌گردد.
- هدف این فرآیند بارگذاری سیستم‌عامل در حافظه و شروع اجرای آن است.
- مراحل اصلی بوت:
  1) CPU شروع به اجرای دستورها از آدرسی ثابت در حافظه می‌کند.
  2) کد اولیه در ROM یا Firmware (BIOS/UEFI) اجرا می‌شود.
  3) این کد برنامه‌ای به نام Bootstrap Loader را پیدا و اجرا می‌کند.
  4) بوت‌لودر کرنل سیستم‌عامل را از دیسک به حافظه می‌آورد.
  5) کرنل اجرا شده و کنترل سیستم را در دست می‌گیرد.
- در سیستم‌های جدید از GRUB Boot Loader برای انتخاب نسخه‌ی کرنل یا سیستم‌عامل استفاده می‌شود.



Operating-System Debugging

- اشکال‌زدایی (Debugging) یعنی پیدا کردن و برطرف کردن خطاها در سیستم‌عامل یا برنامه‌های آن.
- سیستم‌عامل ابزارهایی برای کمک به این کار فراهم می‌کند، مثل:
  - ثبت رویدادها در فایل‌های Log
  - تولید Core Dumpبرای برنامه‌ها (حافظه‌ی برنامه در زمان خطا)
  - تولید Crash Dumpبرای خود سیستم‌عامل هنگام از کار افتادن
- با استفاده از این اطلاعات، برنامه‌نویس می‌تواند علت خطا را بررسی و اصلاح کند.
- اشکال‌زدایی در سطح سیستم‌عامل سخت‌تر از برنامه‌های معمولی است، چون اگر هسته (Kernel) دچار خطا شود، کل سیستم ممکن است از کار بیفتد.
- وقتی در ویندوز صفحه‌ی آبی (Blue Screen of Death) ظاهر می‌شود، سیستم در واقع در حال ایجاد Crash Dumpاست تا بعداً علت خطا بررسی شود.
- قانون معروف کرنیگان :«اشکال‌زدایی دو برابر سخت‌تر از نوشتن کد است؛ بنابراین اگر کدی تا حد ممکن باهوش بنویسید، دیگر به‌اندازه کافی باهوش نیستید تا آن را اشکال‌زدایی کنید!»

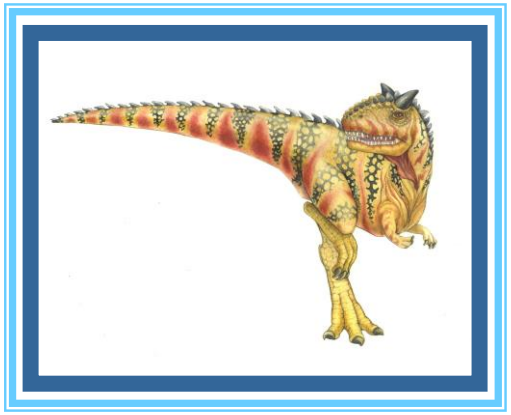


Performance Tuning

- هدف از تنظیم کارایی (Performance Tuning) این است که گلوگاه‌ها (Bottlenecks) و بخش‌های کند سیستم پیدا و اصلاح شوند.
- برای این کار از ابزارهای مانیتورینگ استفاده می‌شود تا مصرف منابع سیستم بررسی شود.
- نمونه ابزارها:
  - در Linux دستورهای `top`، `vmstat`، `iostat`
  - در ویندوز Task Manager و Performance Monitor
- روش کار:
  1) مشاهده وضعیت CPU، حافظه و I/O
  2) شناسایی فرآیندهای سنگین یا غیرعادی
  3) تغییر تنظیمات یا بهینه‌سازی نرم‌افزارها و سرویس‌ها



End of Chapter 2



Silberschatz, Galvin and Gagne ©2013